

Polynomials Trajectory Planner

Preliminary Documentation

Gianluca Castellari

Summer 2026

Abstract

Symbology, nomenclature definitions. Theory of the polynomial profiles.

Contents

1	Vision	1
1.1	Outputs	2
1.2	Topic classification	2
1.3	General rules (apply throughout, unless a requirement says otherwise)	2
2	Requirements	2
3	Data Model	4
3.1	Points	4
3.2	Segments and Trajectories	4
3.3	Polynomial and boundary conditions	5
3.4	Constraints and Blend	5
3.5	State (conceptual view)	6
3.6	How the entities relate	6
4	Assumptions	6
5	Future Topics	6
5.1	Orientation	6
5.2	Time scaling	6
5.3	Multi-axis synchronization	6
5.4	Path vs Trajectory	6
5.5	Numerical Stability	7
5.6	Implementation notes carried over from the original brief	7
6	Success Criteria	7
7	Theory Library Roadmap	7
7.1	Format convention	7
7.2	Pre-publication verification checklist	8
7.3	Out of scope for this roadmap (tracked elsewhere)	9

1 Vision

The goal of this project is to enter in detail into the trajectory planning process, considering polynomial algorithms of different degree.

The degrees to be approached are, at the moment, three, five, seven and nine. Other degrees will be added over time if deemed useful during development (see `04-future-topics.md`).

The project will culminate in an **application** (web or desktop — see the open decision in `01-requirements.md`, item 16) that analytically determines the trajectories discussed throughout the project. It is also intended as a personal learning experience in polynomial calculus and the techniques behind it. Everything produced should therefore be as educational as possible.

1.1 Outputs

Two outputs are expected:

1. **The application** — graphical output and data input, satisfying the technical specifications.
2. **A knowledge reference** — a readable document (or group of documents) / project log, capturing procedures, calculations, considerations, and theory as they are developed. Ideally this log records the steps taken and the theoretical/practical considerations behind each development decision.

1.2 Topic classification

Every requirement in `01-requirements.md` is tagged as one of:

- **[Learning]** — theoretical concepts and procedures aimed at deepening understanding of the topic.
- **[Technical]** — aspects directly involved in the implementation of the application.
- **[Learning & Technical]** — applies to both.

For every tagged item, a written report of the procedures (calculations, considerations) and algorithms used — and, when applicable, of the underlying theory — is expected.

1.3 General rules (apply throughout, unless a requirement says otherwise)

To avoid repeating the same clause on every requirement, the following rules are declared once here and apply globally:

- **Dimensionality:** every analysis, algorithm, or consideration is first developed in one-dimensional (1D) space. Once established, any differences arising when extending the same concept to three-dimensional (3D) space are noted explicitly. This progression (1D → note 3D differences) is the default for all requirements unless stated otherwise.
- **Numerical vs. symbolic:** unless numerical data is strictly necessary for the context, symbolic expressions are preferred over numerical examples.

2 Requirements

Tags: **[Learning]**, **[Technical]**, **[Learning & Technical]** — see `00-vision.md` for definitions. Where a requirement's dimensionality progression (1D → 3D) or symbolic-vs-numeric preference is not restated, the general rules in `00-vision.md` apply.

1. **[Learning & Technical]** Define the symbology to be adopted to clearly and correctly represent polynomial paths in one and three dimensions, including symbols for orienting a solid in three-dimensional space (the latter to be detailed in a later step — see `04-future-topics.md` → Orientation). Two-dimensional notation is intentionally omitted because it is considered a conceptual subset of the three-dimensional case. This is a starting point to be established before other points.
2. Regarding the polynomial of degree 9 (the first to be considered):
 - **[Learning]** Report the canonical form and the Horner form in the time domain; list pros and cons of the two forms.

- **[Learning]** Consider the derivatives up to the sixth, starting from the position algorithm (velocity, acceleration, jerk, snap, crackle, pop); discuss the usefulness of using more or fewer derivatives for trajectory planning.
3.
 - **[Technical]** Report the mathematical procedure to calculate the coefficients of the polynomial, assuming all necessary boundary data has been specified.
 - **[Learning]** List the boundary conditions relevant to each polynomial degree considered in the project, distinguishing between necessary, possible, and redundant conditions for that degree.
 4. **[Learning]** Report the analysis (pros and cons) of using an absolute period vs. a normalised period.
 5. **[Learning & Technical]** Symbolic-vs-numeric preference — see general rule in 00-vision.md.
 6. **[Learning & Technical]** Dimensionality progression for items 2–5 — see general rule in 00-vision.md.
 7. **[Technical]** For a polynomial of degree 9, carry out the following analysis where relevant: procedure and algorithms to determine max/min position, max/min velocity, acceleration, jerk, and snap.
 8. **[Technical]** Procedure and algorithms necessary to impose a velocity limit on a polynomial of degree nine.
 9. **[Technical]** As item 8, but imposing a maximum acceleration.
 10. **[Technical]** As item 8, but imposing a maximum jerk.
 11. **[Learning & Technical]** Dimensionality progression for items 7–10 — see general rule in 00-vision.md.
 12. **[Learning & Technical]** If a constant-speed segment is required within a path originally defined as a degree-nine polynomial, describe the considerations and algorithms needed to satisfy this. The constant-speed segment is specified by its start and end positions within the original path, in absolute terms or percentages (arbitrary choice of convenience). Cover 1D first, then 3D (per general rule).
 13. **[Technical]** Complex trajectories are normally composed of more than one segment (from two to several dozen). The connection between consecutive segments should be specified by: the starting point of the connection (in the segment preceding the common connection point) and the ending point of the connection (in the segment following it) — in absolute terms or percentages. Report the strategy and algorithms involved. Cover 1D first, then 3D (per general rule).
 14. **[Learning & Technical]** The smooth-connection approach from item 13 works correctly except when the path must pass exactly through the endpoints of the specified segments, avoiding a “zero velocity” event. A smooth connection with precise transition points (the segment endpoints) is required instead. Assuming velocity, acceleration, jerk, and snap are zero at the start of the first segment and the end of the last segment, report the considerations and solution strategy for this case, in both 1D and 3D (per general rule).
 15. **[Learning & Technical]** The output resulting from the above points should be summarized in a software application including a graphical representation of the calculated profiles, based on data acquired from an input interface (details to be provided during development). At the moment only the degree-nine polynomial has been considered, but the final version will extend the same considerations/analysis/calculations to degrees seven, five, and three (only the applicable ones per degree). The second output is the project log — a single or multiple readable document(s) reporting notes, considerations, strategies, and theoretical treatments involved in the application.
 16. **[Technical]** — **DECISION: desktop application, C#, WPF.**
 - **Platform:** desktop application (not web).
 - **Language / toolchain:** C#, WPF for the UI, a charting library (e.g. LiveCharts2 / OxyPlot / ScottPlot) for graphical visualization of the computed profiles.
 - **Rationale:** no requirement in this document calls for multi-user access, remote sharing, or cross-device execution — the tool is a single-user local analysis application, so the web’s main

advantages don't apply here. A web stack would additionally require frontend/backend tooling unrelated to trajectory planning, conflicting with the constraint below. A desktop app keeps all calculation logic (coefficients, derivatives, constraints, blending) in one project and one language, which best matches the educational goal of understanding the theory rather than a client/server architecture.

- Constraint (kept from the original brief): the purpose of the project is primarily to deepen knowledge of trajectory planning, not to learn new tools/languages unless strictly necessary for that goal.
- Constraint (kept from the original brief): the project author must remain involved in and aware of the specific content of the project software — the educational aspect must not be sacrificed for development speed, even if this slows progress.
- At this stage no hardware/actuator/kinematics application is considered — theory only (see 03-assumptions.md), even though the natural extension is toward robotic manipulators.
- **Revisit if:** a future requirement emerges for multi-user access, remote/cross-device use, or integration with an online service — none of these apply today.

3 Data Model

This chapter summarizes the data entities involved in the project. Orientation is **not** considered yet in these models (see 04-future-topics.md → Orientation); it will be added later.

This is a reference example for the data potentially involved in the project — it will be revised as the project progresses, as needed.

3.1 Points

```
Point1D {
  x;    // mono-dimensional position value of a single point
}

Point3D {
  x;    // three-dimensional x position value of a single point
  y;    // three-dimensional y position value of a single point
  z;    // three-dimensional z position value of a single point
}
```

3.2 Segments and Trajectories

A segment of a complex trajectory is itself a trajectory:

```
Segment1D {
  startPoint;          // Point1D
  endPoint;            // Point1D
  polynomialDegree;    // chosen among 3, 5, 7, 9 (and eventually other values)
  constraints[];        // settings of constraints applied to the segment
}

Segment3D {
  startPoint;          // Point3D
  endPoint;            // Point3D
  polynomialDegree;    // chosen among 3, 5, 7, 9 (and eventually other values)
  constraints[];        // settings of constraints applied to the segment
}

Trajectory1D {
```

```

    Segment1D[nm];           // array of Segment1D elements
    segmentEndpointRespect;   // yes/no - if yes, segment blending settings become irrelevant
}

Trajectory3D {
    Segment3D[nm];           // array of Segment3D elements
    segmentEndpointRespect;   // yes/no - if yes, segment blending settings become irrelevant
}

```

3.3 Polynomial and boundary conditions

```

Polynomial {
    degree;                  // chosen among three, five, seven, nine, etc.
    coefficients;
    boundaryConditions;
}

BoundaryConditions {
    necessaryConditions[];
    possibleConditions[];
    redundantConditions[];
}

```

`BoundaryConditions` is crucial to the calculation of polynomial profiles of any degree. Its content depends on the polynomial degree being treated: each degree has a set of necessary conditions to determine the polynomial, a set of possible conditions that may be useful in broader formulations, and a set of redundant conditions that exceed what is needed for that degree.

3.4 Constraints and Blend

```

Constraints {
    constraintType;           // time, velocity, acceleration, jerk, snap, etc. (if applicable)
    constraintValue;          // value referred to the selected constraintType
    constantVelocitySelection; // yes/no - activates the constant-velocity part of the segment
    constantVelocityValue;    // applied if constantVelocitySelection = yes
    constantVelocityInitialPos1D; // Point1D - start of the constant-velocity part of the segment in 1D
    constantVelocityEndPos1D;    // Point1D - end of the constant-velocity part of the segment in 1D
    constantVelocityInitialPos;  // Point3D - start of the constant-velocity part of the segment
    constantVelocityEndPos;      // Point3D - end of the constant-velocity part of the segment
}

Blend {
    nextSegmentBlendingPos1D;   // Point1D - where blending with the next segment starts in 1D
    previousSegmentBlendingPos1D; // Point1D - where blending with the previous segment ends in 1D
    nextSegmentBlendingPos;     // Point3D - where blending with the next segment starts
    previousSegmentBlendingPos;  // Point3D - where blending with the previous segment ends
}

```

Design note (confirmed distinction): `Constraints` and `Blend` are deliberately distinct concepts, kept separate: - **Constraints** describes kinematic limits/settings applied *within* a single segment (velocity/acceleration/jerk/snap limits, and the optional constant-velocity sub-part of the segment, via the dedicated 1D or 3D positions). - **Blend** describes the geometric continuity *between* adjacent segments — the *raccordo* (smoothing) at the junction points, via the dedicated 1D or 3D blending positions.

Relevant to requirements 12–14 in 01-requirements.md.

3.5 State (conceptual view)

The **State** element is an elementary entity of the trajectory concept: it specifies an endpoint (initial or final, relative to a single segment) and its relevant details.

```
State1D {  
    position; velocity; acceleration; jerk; snap;  
}
```

The same concept applies to **State3D**. **State** may be more relevant than **Point** when approaching the framework's motion-planning task.

3.6 How the entities relate

- **Constraints** and **Blend** are project-specific requirements — their settings/values are internal parts of the whole project (see open note above).
- **Trajectory** is the top-level container that encompasses the entire data structure (all elements above, in terms of selections and values) leading to the determination of the overall movement profile — the purpose of the project.

4 Assumptions

1. Only theoretical trajectory generation.
2. No actuator dynamics.
3. No kinematic constraints.
4. Single-axis analysis before multi-axis analysis.
5. Symbolic treatment preferred over numerical examples.

5 Future Topics

Not in the current scope, but tracked for later consideration.

5.1 Orientation

- Euler angles.
- Quaternions.
- Rotation matrices.

5.2 Time scaling

- Trajectory stretching.
- Trajectory compression.

5.3 Multi-axis synchronization

- Synchronous end-time.
- Master axis.
- Slave axes.

5.4 Path vs Trajectory

- Path = geometric definition.
- Trajectory = path + time law.

5.5 Numerical Stability

- Conditioning.
- Stability.
- Floating point errors.

5.6 Implementation notes carried over from the original brief

- Polynomial degrees other than three, five, seven and nine may be added if deemed useful during project development.
- Application type decided: desktop application in C# with WPF (see `01-requirements.md`, item 16).
- Readable documentation for learning is a hard requirement (the author’s own stated need), not optional polish.
- Open question: derivatives usefulness — up to which degree is it actually useful to compute/use them?
- Orientation of a solid in three-dimensional space (see Orientation above).
- Technical tools for app development decided: C# for the calculation/application code, WPF for the UI, and a charting library chosen during implementation — see `01-requirements.md`, item 16.

6 Success Criteria

A project is considered successful if:

- The mathematical theory is completely documented.
- Every algorithm is explained.
- Every software decision is justified.
- The generated application can calculate polynomial trajectories.
- The generated application can visualize the results graphically.
- A complete educational project log is produced.
- The author can understand the theory and implementation process.

Suggestion (not part of the original brief): the criteria above are qualitative. If measurable acceptance criteria would help track progress with Trae (e.g. “the app computes and plots a degree-9 profile from 10 boundary conditions in under 1s”), consider adding a short “Measurable criteria” section here once the technical stack (item 16) is decided.

7 Theory Library Roadmap

This is the working checklist for the “personal library” of theory/technical documents to be produced with the AI IDE, one requirement-cluster at a time. Each entry maps to the requirement(s) it covers in `01-requirements.md`, so coverage can be checked off as documents are produced.

Suggested per-document template (for consistency across the library): short intro / scope → symbols used (referencing `SymbolsAndNomenclature`) → theory / derivation → algorithm or procedure → worked considerations (symbolic, per `03-assumptions.md`) → open questions / links to related documents.

7.1 Format convention

- **Source format:** every document in this library is written in **Markdown** (`.md`), consistent with the specification files (`00–05`).
- **Mathematical notation:** formulas and equations are written as LaTeX syntax within the Markdown: inline math uses single delimiters like $x(t)$, while display equations on their own line use double delimiters like
$$x(t) = \sum_{i=0}^9 a_i t^i$$
. Math delimiters are never wrapped in backticks. This keeps documents renderable in-editor (MathJax/KaTeX) and diffable in version control.

- **Reading / archival output:** when a readable/printable version of the library is needed, it is generated from the Markdown sources via `pandoc` (LaTeX engine) into PDF — the Markdown+LaTeX-inline files remain the single source of truth; PDF is a generated artifact, never edited directly.
- **Not used as a working/source format:** LaTeX (.tex) as primary source, Word (.docx), ODT, plain .txt. See discussion in chat for the rationale.

7.2 Pre-publication verification checklist

Before considering any document in this library “finished”, verify it with the following steps:

1. Compile it: `pandoc <file>.md -o <file>.pdf --pdf-engine=xelatex -V mainfont="<a system font>" -V geometry:margin=2.5cm`
2. **Check inline math specifically** (formulas embedded in running text, not standalone display equations on their own line) — this is where syntax errors are most likely to hide, since display-equation blocks tend to be visibly broken (and so get caught immediately) while inline math can silently render as literal text instead of a formula.
3. Confirm backticks are only used around genuine code/filename references, never around $\dots$ math delimiters — inline math must be single $\dots$, with no surrounding backticks (backtick-wrapped $\dots$ renders as literal text instead of a formula).
4. If the PDF build fails with a missing LaTeX package (e.g. `lmodern.sty` not found), install the missing package via the local TeX distribution (MiKTeX installs it automatically on Windows; `tlmgr install <package>` on TeX Live) — or work around it by forcing a system font with `--pdf-engine=xelatex -V mainfont="<font name>".`
5. **In tables, never write a multi-symbol list as a single inline math block** (e.g. x_0, x_f, v_0, v_f) — LaTeX cannot line-wrap inside one math box, so it silently overflows into the next column instead of wrapping. Write each symbol as its own inline math, comma-separated in plain text (e.g. x_0 , x_f , v_0 , v_f), so the cell can wrap normally.
6. **Respect heading hierarchy — never place two consecutive headings at the same level when the second is logically a subsection of the first** (e.g. a `##` “1D Blending Problem” section immediately followed by a `##` “Nominal Consecutive Segments” that is actually part of it). The second heading should be one level deeper (`###`).

#	Document	Covers (requirements)	Status
1	<code>SymbolsAndNomenclature</code>	Req. 1 — symbology for 1D/3D polynomial paths and solid orientation (Euler angles, quaternions, rotation matrices — notation reserved now, full treatment later per <code>04-future-topics.md</code>)	Draft v1
2	<code>NinthDegreePolynomialTheory</code>	Req. 2 — canonical & Horner form, derivatives up to the 6th (velocity $\rightarrow$ pop), pros/cons discussion	Draft v1
3	<code>BoundaryConditions</code>	Req. 3 (learning part) — full list of boundary conditions needed to determine the polynomial	Draft v1

#	Document	Covers (requirements)	Status
4	<code>PolynomialCoefficientDerivation</code>	Req. 3 (triboninical part) + Req. 4 — coefficient-calculation procedure; absolute vs. normalised period analysis	Draft v1
5	<code>TrajectoryConstraints</code>	Req. 7–10 — max/min posi- tion/velocity/acceleration/jerk/snap analysis; imposing veloc- ity/acceleration/jerk limits	Draft v1
6	<code>ConstantVelocitySegment</code>	Req. 12 — constant-speed segment within a degree-9 path (start/end position, absolute or percentage)	Draft v1
7	<code>BlendingSegments</code>	Req. 13 — connecting consecutive segments without a pass-through constraint at the join	Draft v1
8	<code>TrajectoryPassingThroughConstraintPoints</code>	Req. 14 — connection with precise transition points (zero veloc- ity/acceleration/jerk/snap at path start/end)	Draft v1

Note on item 6: this document was not in the original list of documents discussed — it covers requirement 12 (constant-velocity segment), which is distinct both from `TrajectoryConstraints` (general kinematic limits) and from `BlendingSegments`/`TrajectoryPassingThroughConstraintPoints` (continuity *between* segments, per the `Constraints` vs. `Blend` distinction in `02-data-model.md`). It is placed here because it logically follows the general constraints treatment and precedes the inter-segment blending topics.

7.3 Out of scope for this roadmap (tracked elsewhere)

- Requirement 15 (the application itself, graphical interface, extension to degrees 3/5/7) is not a theory document — it is the implementation phase in `src/`, to start once the library above (or at least items 1–5) is in reasonably solid shape.
- Requirements 6 and 11 (1D → 3D progression) are not separate documents — per the general rule in `00-vision.md`, each document above covers 1D first and notes 3D differences inline.
- Future Topics (orientation in full, time scaling, multi-axis sync, numerical stability) remain in `04-future-topics.md` until promoted to their own roadmap entries.